

ইন্টিগ্রেশন ব্রিফ

# ERPNext ⇄ WooCommerce ইন্টিগ্রেশন ব্রিফ

WooCommerce REST API দিয়ে প্রোডাক্ট, অর্ডার ও কাস্টমার  
সিঙ্ক্রোনাইজেশন

যাঁর জন্য

ERPNext ইন্টিগ্রেশন  
ডেভেলপার

যাঁর পক্ষ থেকে

Dressup ·  
WooCommerce স্টোর

তারিখ

16 সেপ্টেম্বর  
2026

স্ট্যাটাস

একমাত্র নির্ভরযোগ্য  
ডকুমেন্ট

**ভাষা নিয়ে একটা কথা।** এখানে ব্যাখ্যাগুলো বাংলায়, কিন্তু সব code, field name, API path, HTTP status আর টেকনিক্যাল টার্ম ইংরেজিতেই রাখা হয়েছে — কারণ ওগুলো ছবছ ওভাবেই লিখতে হবে, অনুবাদ করলে ভুল হবে। সংখ্যাও ইংরেজি অঙ্কে (1, 2, 3) রাখা, যাতে ইংরেজি ভার্সনের সাথে সেকশন নম্বর মিলিয়ে পড়তে পারেন। এটি ইংরেজি ভার্সনের আক্ষরিক অনুবাদ নয় — একই কথা বাংলায় পরিষ্কার করে বলা। দুটোর কোথাও অমিল মনে হলে ইংরেজি ভার্সনটাই চূড়ান্ত ধরবেন, আর আমাদের জানাবেন।

## সূচিপত্র

### আগে এটুকু পড়ুন

1. **আমরা যা চাই, এক প্যারাগ্রাফে** — পুরো ব্যাপারটা এক প্যারাগ্রাফে।
2. **আপনার কাছে প্রথম যা চাই: আপনার সার্ভারের IP address** — আজকেই করুন। একটা কমান্ড, এক লাইন আউটপুট — এতেই বাকি সব খুলে যাবে।
3. **চারটি শর্ত** — যে চারটি শর্তের উপর এই ইন্টিগ্রেশন দাঁড়িয়ে আছে।
4. **আগে নিজের একটা stock WooCommerce-এ করে দেখান** — রেডিনেস চেক। আমাদেরটা পাওয়ার আগে নিজের WooCommerce-এ 10টি টেস্ট।
5. **আপনার কাজের ধরন নিয়ে আমাদের প্রত্যাশা** — AI দিয়ে কাজ করায় আপত্তি নেই — শুধু একটা নির্দিষ্ট ভুল বন্ধ করতে হবে।

### আমরা যা বানাচ্ছি

6. **আমাদের কাছ থেকে আপনি যা পাবেন** — এবং এটুকুই — স্টেজিং URL আর API credential। ঠিক দুটো জিনিস।
7. **স্কোপ** — কোনটা এখন করব, কোনটা করব না, কোনটা আপাতত হাত দেবেন না।
8. **দুই সিস্টেম কীভাবে একই জিনিস চিনবে** — সবচেয়ে গুরুত্বপূর্ণ সেকশন। SKU, ID, আর ম্যাপিং কার কাছে থাকবে।
9. **Cloudflare** — আমরা এটা কীভাবে সামলাব — এটা আমাদের দিকের কাজ। আপনার জন্য কোনো বাধা নয়।
10. **কোন কাজটা কোন ক্রমে করবেন** — 10টি ধাপ, ক্রম অনুযায়ী। Webhook কেন পরে আসে, সেটাও বলা আছে।

### তিনটি ডেটা ফ্লো

11. **প্রোডাক্ট** — ERPNext → WooCommerce — কোন ফিল্ড, ব্যাচিং, প্রোডাক্ট বন্ধ করা।
12. **অর্ডার** — WooCommerce → ERPNext — Polling, idempotency, টাকার হিসাব।
13. **কাস্টমার** — অ্যাকাউন্টধারী আর গেস্ট — দুটো আলাদাভাবে।

### চালু রাখা

14. **চালানোর নিয়ম** — Retry, rate limit, credential, logging, reconciliation।
15. **অ্যাকসেসপেটেন্স চেকলিস্ট** — সাইন-অফের আগে আমরা যে ঘরগুলো আসলেই টেস্ট করব।
16. **কিছু কাজ না করলে** — সমস্যা ঘুরিয়ে না গিয়ে আমাদের কীভাবে জানাবেন।

# 1 আমরা যা চাই, এক প্যারাগ্রাফে

ERPNext আর আমাদের WooCommerce স্টোরের মধ্যে প্রোডাক্ট, অর্ডার আর কাস্টমার — এই তিনটা জিনিস সিলেক্স রাখা। এটা আপনি করবেন ERPNext থেকে WooCommerce-এর স্ট্যান্ডার্ড REST API কল করে, আমাদের দেওয়া consumer key আর secret ব্যবহার করে। আমাদের স্টোরে আপনি কিছুই ইনস্টল করবেন না। সব লজিক, সব শিডিউলিং, সব ম্যাপিং আর সব ভারী কাজ চলবে আপনার দিকে, ERPNext-এর ভেতরে। আমাদের স্টোর একটা সাধারণ WooCommerce স্টোর যেটা API কল-এর জবাব দেয় — এর বেশি কিছু না। পুরোনো প্লাগিন আর যা যা করত (pricing rule, loyalty point, discount, dashboard) সেসব এখনকার ক্লোপের বাইরে, ওগুলো নিয়ে পরে আলাদা করে কথা হবে।

## 2 আপনার কাছে প্রথম যা চাই: আপনার সার্ভারের IP address

আর সব কিছুর আগে, আপনার ERPNext সার্ভারে এই কমান্ডটা চালান আর আউটপুটটা আমাদের পাঠান।

```
curl -s https://www.cloudflare.com/cdn-cgi/trace | grep '^ip='
```

এটা এক লাইন প্রিন্ট করবে, যেমন `ip=85.190.254.51`। আমাদের এটুকুই দরকার।

### এটা সবার আগে কেন চাইছি

আমাদের স্টোর Cloudflare-এর পেছনে বসানো। Cloudflare-এর প্রোটেকশন হয় পুরোটা চালু, নয় পুরোটা বন্ধ — "আক্রমণকারীদের আটকাও, কিন্তু এই একটা API client-কে ছেড়ে দাও" বলার মতো কোনো সেটিং নেই। আপনার সার্ভারকে ঢুকতে দেওয়ার একমাত্র উপায় হলো একটা custom rule-এ তার IP address স্পষ্টভাবে allow-list করা।

Address না জানা পর্যন্ত আমরা ওই rule লিখতে পারব না। আর ওই rule না থাকলে আমরা যে credential-ই দিই, সেটা কাজ করবে না — আপনার request WooCommerce পর্যন্ত পৌঁছানোর আগেই challenge থাকবে বা block হয়ে যাবে। সেই অবস্থায় বসে ডিবাগ করাটা আমাদের কারোরই কাজে লাগবে না।

তাই ক্রমটা এরকম:

1. আপনি ওই কমান্ডের আউটপুট আমাদের পাঠাবেন।
2. আমরা allow-list rule বানিয়ে ডেভিকেটেড স্টেজিং স্টোরটা দাঁড় করাব — 24 থেকে 48 ঘণ্টা।
3. আমরা WhatsApp গ্রুপে URL আর API credential দেব।
4. আপনি কানেক্ট করবেন, আর প্রথমবারেই কাজ করবে।

### দুটো ব্যাপার খেয়াল রাখবেন

- কমান্ডটা ERPNext সার্ভারেই চালাবেন। DNS record দেখে IP বলে দেবেন না। একটা সার্ভার যে address থেকে পাঠায় সেটা প্রায়ই সেই address না যেটাতে সে গ্রহণ করে — NAT gateway, load balancer আর container networking-এর কারণে

এটা হয়। ভুল address পাঠালে আমাদের ড্যাশবোর্ডে rule-টা দেখতে একদম ঠিকঠাক লাগবে, অথচ আপনার request block হতেই থাকবে।

- **একাধিক সার্ভার বা worker থাকলে** প্রত্যেকটায় চালান আর সবগুলো পাঠান, অথবা সবগুলোকে কভার করে এমন একটা CIDR range পাঠান।

আপনার হোস্টিং যদি স্থায়ী outbound IP দিতেই না পারে, তাহলে পরে না বলে এখনই বলুন — shared secret header দিয়ে একটা বিকল্প ব্যবস্থা আছে, সেটা কোনো সমস্যা না, কিন্তু rule বানানোর আগে আমাদের জানতে হবে।

**এই ডকুমেন্টের আর কোনো কিছু এটার জন্য আটকে নেই।** §4-এর সব কাজ আপনার নিজের WooCommerce ইনস্টলেই হবে। IP-টা পাঠিয়ে দিন, তারপর পড়তে থাকুন আর বানানো শুরু করুন।

## 3 চারটি শর্ত

এগুলো পছন্দ-অপছন্দের ব্যাপার না। এগুলোই ইন্টিগ্রেশনের শর্ত।

### শর্ত 1 — কোনো প্লাগিন নয়। আমাদের সার্ভারে কোনো কোড নয়।

আমরা কোনো plugin, must-use plugin, snippet, custom endpoint বা theme edit — কিছুই ইনস্টল করব না। আপনার কোনো কোড আমাদের ইনফ্রাস্ট্রাকচারে চলবে না।

আপনার যা যা দরকার তার সবই WooCommerce-এর REST API-তে আগে থেকেই আছে। যদি সত্যিই মনে করেন আমাদের দিকে কোড ছাড়া কোনো একটা জিনিস কোনোভাবেই সম্ভব না, তাহলে কিছু বানানোর আগে আমাদের বলুন সেটা কী আর কেন — ইনস্টল করার জন্য প্লাগিন পাঠাবেন না।

### শর্ত 2 — Cloudflare চালু থাকবে, আর সেটা আমরাই সামলাব।

আমাদের স্টোর Cloudflare-এর পেছনে। এটা আপনার ঘুরে যাওয়ার মতো কোনো জিনিস না, আর এটা আপনার জন্য কোনো বাধাও না — এটা আমাদের দিকের সেটআপ, স্টেজিং credential দেওয়ার আগেই আমরা এটা ঠিক করে রাখব।

তবে বন্ধ না করে rule দিয়ে কেন করছি, সেটা বলে রাখা দরকার: Cloudflare-কে একটা টগল দিয়ে অল্প অল্প করে ঢিলা করা যায় না। বন্ধ করা মানে পুরোটাই বন্ধ, আর আপনার ট্রাফিকের জন্য বন্ধ করা মানে সবার জন্য বন্ধ করা — যত scanner, credential-stuffing bot আর scraper এখন এটা আটকে দিচ্ছে, সবার জন্য খুলে দেওয়া। তাই তার বদলে আমরা একটা **সরু custom rule** বসাই, যেটা শুধু আপনার ট্রাফিক চিনে সোজা ছেড়ে দেয়, আর বাকি সবকিছুর উপর প্রোটেকশন যেমন ছিল তেমনই থাকে।

**আপনার কাছ থেকে শুধু দরকার সেই IP address, যেটা দিয়ে আপনার ERPNext সার্ভার বাইরে কল করে** (কয়েকটা থাকলে CIDR range দিলেও চলবে)। ওটুকু পাঠিয়ে দিন, বাকিটা আমাদের কাজ। বিস্তারিত §9-এ।

কাজের একটা কথা: আমাদের কাছে আসবেন স্বাভাবিক hostname দিয়ে, সরাসরি আমাদের origin IP ধরে না। সরাসরি origin-এ গেলে আপনার জন্য বানানো rule-টাই বাদ পড়ে যায়, আর আমাদের origin address বদলালেই ভেঙে পড়ে — তাই এটা সাধারণত একবার কাজ করে, তারপর মাসখানেক পরে অদ্ভুতভাবে ফেল করে।

### শর্ত 3 – ভারী কাজটা আপনার।

আপনি pull করবেন। আপনি push করবেন। আপনি শিডিউল করবেন। আপনি retry করবেন। আপনি মিলিয়ে দেখবেন। দুই সিস্টেমের মধ্যে ম্যাপিং আপনি রাখবেন। আপনি মনিটর করবেন, আর ভেঙে গেলে আমাদের জানাবেন।

আমাদের দিক ঠিক একটা কাজ করে: সঠিক প্রোডাক্ট, অর্ডার আর কাস্টমার ডেটা ধরে রাখা এবং একটা স্ট্যান্ডার্ড API দিয়ে সেটা পরিবেশন করা। আমরা কোনো sync engine চালাচ্ছি না, আপনার হিসাব-নিকাশ জমা রাখছি না, আর আপনার দিকের নোট রাখার জায়গা দিতে আমাদের প্রোডাক্টে custom field যোগ করছি না। ইন্টিগ্রেশনের state সেই সিস্টেমের, যে সিস্টেমটা ইন্টিগ্রেট করছে — অর্থাৎ আপনার।

### শর্ত 4 – Checkout-এ কিছু ছোঁয়া যাবে না।

কাস্টমার checkout করার সময় কোনো কোড, কোনো hook, কোনো API কল — কিছু চলবে না। পুরোনো প্লাগিনটা প্রতিটা page load-এ ERPNext-এর জবাবের জন্য আমাদের checkout-কে 12 সেকেন্ড পর্যন্ত বসিয়ে রাখত, আর ERPNext ধীর থাকলে কাস্টমারের কাছ থেকে ভুল দাম নেওয়া হতো। ঠিক এই সমস্যাটাই আমরা বন্ধ করছি, আর পুরো জিনিসটা নতুন করে লেখার মূল কারণই এটা।

আপনার ইন্টিগ্রেশন ঘটনা ঘটে যাওয়ার পরে, আপনার নিজের শিডিউলে পড়বে আর লিখবে। ERPNext এক ঘণ্টা বন্ধ থাকলেও আমাদের স্টোর বিক্রি করতে থাকবে, আর আপনি ফিরে এসে পিছিয়ে থাকা কাজটা ধরে ফেলবেন।

## 4 আগে নিজের একটা stock WooCommerce-এ করে দেখান

এটাই রেভিনেস চেক। এটা পাস না করা পর্যন্ত আমাদের স্টেজিং credential পাবেন না।

আপনার ইন্টিগ্রেশনটা আগে আপনার নিজের একটা পরিষ্কার, সাম্প্রতিক WooCommerce ইনস্টলের বিপরীতে বানান — লোকাল, সস্তা একটা VPS, wp-env, LocalWP, যা খুশি। ওখানে নিজের API key বানান। পুরো জিনিসটা আগে ওখানে কাজ করতে হবে।

এটা কোনো ফাঁদ না। এটা আমাদের দুজনকেই বাঁচায়:

- আপনি এমন একটা স্টোরে ডিবাগ করছেন যেখানে কিছু ভাঙলে কোনো ক্ষতি নেই।
- যত সমস্যায় পড়বেন সেগুলো আপনার সমস্যা, আমাদের সাথে সাপোর্ট থ্রেড না।
- আমাদের স্টেজিং স্টোরে যখন আসবেন, তখন হাতে এমন কিছু নিয়ে আসবেন যেটা ইতিমধ্যে কাজ করে — তখন আমরা ইন্টিগ্রেশন টেস্ট করব, API শেখাব না।

### "হয়ে গেছে" বলতে কী বোঝায়

নিচের টেস্টগুলো নিজের stock WooCommerce-এ চালান আর আসল আউটপুট আমাদের পাঠান — কী হওয়ার কথা তার বর্ণনা না, টার্মিনালের সত্যিকারের আউটপুট:

| #  | টেস্ট                                                                  | পাস করলে যা দেখা যাবে                                                            |
|----|------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| 1  | নিজের key দিয়ে GET /wp-json/wc/v3/system_status                       | 200                                                                              |
| 2  | এক কলে POST /products/batch দিয়ে 100টা প্রোডাক্ট তৈরি                 | 100টা তৈরি হয়েছে; response-এর প্রতিটা element-এ error key আছে কি না দেখা হয়েছে |
| 3  | পুরো প্রোডাক্ট sync দ্বিতীয়বার চালান                                  | দ্বিতীয়বারে কোনো পরিবর্তন হয়নি                                                 |
| 4  | শুধু একটা প্রোডাক্টের দাম আর stock আপডেট করুন                          | status, name আর description অপরিবর্তিত                                           |
| 5  | 3টা টেস্ট অর্ডার দিন, আপনার poller চালান                               | ঠিক 3টা Sales Order                                                              |
| 6  | সাথে সাথেই poller আবার চালান                                           | এখনও ঠিক 3টা। 6টা না।                                                            |
| 7  | একটা webhook রেজিস্টার করুন, fire করুন, একই delivery 5 বার replay করুন | ঠিক 1টা Sales Order                                                              |
| 8  | ইচ্ছে করে ভুল signature দিয়ে একটা webhook পাঠান                       | 401, আর কিছুই লেখা হয়নি                                                         |
| 9  | আপনার client-কে এমন URL-এ পাঠান যেটা 500 দেয়                          | 1/2/4/8/16 সেকেন্ড ব্যাক-অফ করে, তারপর থামে                                      |
| 10 | আপনার client-কে এমন URL-এ পাঠান যেটা 404 দেয়                          | সাথে সাথে থামে। retry করে না।                                                    |

আমাদের পাঠাবেন: কোন WooCommerce ভার্সনে টেস্ট করেছেন, WordPress-এর ভার্সন, আর উপরের প্রতিটা টেস্টের আউটপুট।

## তারপর বলুন কোন স্টেজিং অপশনটা চান

| অপশন                                | কী জিনিস                                                                                                                             | কত সময় লাগবে                       |
|-------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| A — ডেভিকেটেড ERP sync স্টেজিং সাইট | <a href="https://erpsync.dressup.com.bd/">https://erpsync.dressup.com.bd/</a> — এই ইন্টিগ্রেশনের জন্যই আমরা আলাদা করে দাঁড় করাচ্ছি। | আপনার IP পাওয়ার 24-48 ঘণ্টার মধ্যে |
| B — নতুন একটা ছবছ 1:1 রেল্লিকা      | আমাদের বর্তমান লাইভ প্রোডাক্ট লিস্টের ছবছ নকল আরেকটা স্টেজিং সাইট। A-এর পাশাপাশি লাগলে দেওয়া যাবে।                                  | আগে থেকে জানাতে হবে                 |

অপশন A-ই ডিফল্ট। সুনির্দিষ্ট কারণ থাকলে তবেই B চাইবেন।

[erpsync.dressup.com.bd](https://erpsync.dressup.com.bd/) -এর অবস্থা: এই স্টোরটা আমরা এই ইন্টিগ্রেশনের জন্যই বানাচ্ছি, আর সময় গোনা শুরু হবে আপনি IP পাঠানোর পর থেকে (§2)। ওটা পাওয়ার পর স্টোরের ডেটা বসাতে আর আপনার allow-list rule জায়গামতো দিতে আমাদের 24 থেকে 48 ঘণ্টা লাগবে, তারপর URL আর API credential WhatsApp গ্রুপে দিয়ে দেব।

ক্রমটা আমরা ইচ্ছে করেই এভাবে রেখেছি। rule বসানোর আগে credential দিয়ে দিলে সেটা কাজ করত না — আপনার request WooCommerce-এ পৌঁছানোর আগেই Cloudflare-এ আটকে যেত, আর কেন আটকাচ্ছে বুঝতে আমাদের দুজনেরই একটা দিন নষ্ট হতো। স্টোর বানানোর সময়েই rule-টা ভেতরে ঢুকিয়ে দিলে credential হাতে পাওয়ামাত্রই কাজ করবে।

**এর মধ্যে আপনি বসে থাকবেন না।** উপরের রেডিনেস চেকের সব কাজ *আপনার নিজের* stock WooCommerce ইনস্টলে হবে, আমাদেরটায় না। এখনই শুরু করুন। ওই 10টা টেস্ট পাস করাতে করাতে আমাদের স্টেজিং স্টোর আপনার জন্য তৈরি হয়ে বসে থাকবে — আর আগে তৈরি হয়ে গেলে আপনি না আসা পর্যন্ত ওটা এমনিই পড়ে থাকবে।

স্টেজিং সাইটটাও প্রোডাকশনের মতোই Cloudflare-এর পেছনে চলবে, যাতে ওখানে যা বানাবেন লাইভে যাওয়ার দিনেও সেটা একইভাবে আচরণ করে। আপনার অ্যাক্সেস rule বসানোটা ওই 24-48 ঘণ্টার কাজেরই অংশ — এটা আমাদের দিকে হবে, আপনার কাছ থেকে শুধু outbound IP-টা লাগবে (§6)। বিস্তারিত জানতে চাইলে §9 দেখুন।

## 5 আপনার কাজের ধরন নিয়ে আমাদের প্রত্যাশা

আমরা জানি আপনারা AI-first কাজ করছেন, বেশিরভাগ ইমপ্লিমেন্টেশন এজেন্ট দিয়ে করাচ্ছেন। তাতে আপত্তি নেই — আপনার দোকান কীভাবে চালাবেন সেটা আমরা বলতে যাব না। কিন্তু এই ধরনের কাজের একটা নির্দিষ্ট, সুপরিচিত ফাঁক আছে যেটা বন্ধ করতেই হবে, কারণ সেই ফাঁকের ধাক্কাটা আমরা আগেই খেয়েছি।

**AI মডেল উত্তর দেয় তার training data থেকে, যেটা অতীতের একটা নির্দিষ্ট সময়ে আটকে আছে। WooCommerce আটকে নেই।**

আগের ইন্টিগ্রেশনটা বানানো হয়েছিল **WooCommerce 8.5**-এর বিপরীতে। এখনকার ভার্সন **11.1.0**। এর মাঝে WooCommerce, High-Performance Order Storage-কে ডিফল্ট করেছে (8.2) আর block-based Cart ও Checkout-কে ডিফল্ট করেছে (8.3)। ফল যা হলো: কোডটা দেখতে ঠিক লাগত, চোখ বুলিয়ে রিভিউ করলেও পাস করত, অথচ বর্তমান স্টোরে চলত না — কারণ যে hook-গুলোর উপর সেটা নির্ভর করছিল, সেগুলো ওখানে আর চলেই না।

এটা যোগ্যতার সমস্যা না। মডেলের আউটপুট বর্তমান ডকুমেন্টেশনের সাথে কেউ মিলিয়ে না দেখলে এটাই হয়। তাই:

### যা যা করতেই হবে

1. প্রতিটা endpoint, field name আর parameter লাইভ ডকুমেন্টেশনের সাথে মিলিয়ে নিন —

<https://developer.woocommerce.com/docs/apis/rest-api/v3/> — হাড়ার আগে। মডেলের মনে থাকা জিনিসের সাথে না। আপনার এজেন্ট যদি কোনো API ডিটেইলের জন্য বর্তমান একটা ডকুমেন্টেশন পেজ দেখাতে না পারে, তাহলে ওই ডিটেইলটা যাচাই হয়নি ধরে নিন।

2. কোন WooCommerce ভার্সনে বানিয়েছেন আর টেস্ট করেছেন, ডেলিভারির সময় সেটা লিখিতভাবে জানান। আমরা আমাদের চলতি ভার্সনের সাথে মিলিয়ে দেখব।

3. এই কাজের জন্য কোনো এজেন্টকে WordPress প্লাগিন বানাতে দেবেন না। শর্ত 1 দেখুন। আপনার মডেলের বের করা সমাধানে যদি এমন কোনো .php ফাইল থাকে যেটা আমাদের সার্ভারে যাবে, তাহলে মডেল ভুল সমাধানের দিকে চলে গেছে — ওটাই আগের পদ্ধতি ছিল, আর সেটা আমরা স্পষ্টভাবেই আর করছি না।

4. Hook-ভিত্তিক উত্তর দেখলেই সতর্ক হোন। মডেল যদি add\_action , add\_filter , woocommerce\_before\_checkout\_form , woocommerce\_cart\_calculate\_fees বা এই ধরনের কিছু বলে, তাহলে সে প্লাগিন লেখার দিকে চলে গেছে। এই প্রজেক্টে শুধু HTTP request। এখানে কোনো hook নেই।



5. **টেস্টগুলো আসলে চালান। বর্ণনা করবেন না।** এজেন্ট টেস্ট না চালিয়েই খুব আত্মবিশ্বাসের সাথে বলবে "পাস করেছে"। রেডিনেস চেকের প্রতিটা আইটেমের জন্য আসল আউটপুট, আসল সংখ্যাসহ পেস্ট করতে হবে।
6. **Dependency pin করুন আর lockfile commit করুন।** ছয় মাস পরেও আপনার build হুবহু আবার বানানো যেতে হবে।

## এক লাইনের যাচাই

প্রস্তাবিত সমাধানে যদি আমাদের সার্ভারে কিছু ইনস্টল করা লাগে, বা কোনো WordPress hook লাগে — তাহলে মডেল প্লাগিন বানানোর দিকে চলে গেছে। এই প্রজেক্টে শুধু HTTP request, আর কিছু না।

## 6 আমাদের কাছ থেকে আপনি যা পাবেন — এবং এটুকুই

রেডিনেস চেক পাস করার পর ঠিক দুটো জিনিস:

1. **স্টেজিং স্টোরের URL** — <https://erpsync.dressup.com.bd/> এই ইন্টিগ্রেশনের জন্য ডেভিকেটেড একটা স্টোর। আপনি IP পাঠানোর 24-48 ঘণ্টার মধ্যে তৈরি (§2) — স্টোর বানানোর সময়েই আপনার allow-list rule ভেতরে বসিয়ে দিচ্ছি, তাই credential হাতে পাওয়ামাত্রই কাজ করবে। WhatsApp গ্রুপে কনফার্ম করব। লাইভ ক্যাটালগের 1:1 রিপ্লিকাও লাগলে আগে থেকে জানিয়ে চেয়ে নেবেন।
2. **একটা WooCommerce REST API consumer key আর secret**, যেটা WooCommerce নিজেই তৈরি করবে  
*WooCommerce → Settings → Advanced → REST API* থেকে, **Read/Write** পারমিশনসহ।

```
Consumer key: ck_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
Consumer secret: cs_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

ওই credential দিয়ে প্রোডাক্ট, অর্ডার আর কাস্টমারে পুরো read আর write অ্যাক্সেস পাবেন। আপনার যা যা দরকার তার সবই এতে আছে। দ্বিতীয় কোনো সিস্টেম নেই, বাড়তি কোনো endpoint নেই, আলাদা কোনো app নেই, আর বিশেষ কোনো অ্যাক্সেস চাওয়ারও কিছু নেই।

প্রোডাকশন credential দেওয়া হবে **স্টেজিং সাইন-অফ হওয়ার পরেই**। সব কিছু স্টেজিং-এর বিপরীতে বানাবেন।

আপনার কাছ থেকে আমাদের যা দরকার:

1. **আপনার রেডিনেস চেকের ফলাফল** — টেস্টের আউটপুট আর কোন WooCommerce ভার্সনে বানিয়েছেন।
2. **আপনার outbound IP বা CIDR** — এটা §2-এ বলা হয়েছে। এখনও না পাঠিয়ে থাকলে, আপনার স্টেজিং স্টোর ঠিক এই একটা জিনিসের জন্যই আটকে আছে।
3. **আপনার HTTPS webhook URL** — পরে লাগবে, ধাপ 7-এ। তার আগে দরকার নেই।



## 7 স্কোপ

### এখন যা যা স্কোপে আছে

| # | কী                                                | কোন দিকে              | আসল উৎস (source of truth) |
|---|---------------------------------------------------|-----------------------|---------------------------|
| 1 | প্রোডাক্ট — নাম, দাম, stock                       | ERPNext → WooCommerce | ERPNext                   |
| 2 | অর্ডার — বিক্রি, line item, টোটাল, ঠিকানা         | WooCommerce → ERPNext | WooCommerce               |
| 3 | কাস্টমার — অ্যাকাউন্টধারী আর তাঁদের ঠিকানা        | WooCommerce → ERPNext | WooCommerce               |
| 4 | অর্ডার স্ট্যাটাস — fulfilled, cancelled, refunded | ERPNext → WooCommerce | ERPNext                   |

হ্যাঁ, WooCommerce কাস্টমার সিঙ্ক্রোনাইজেশন সাপোর্ট করে — আমরা যাচাই করে দেখেছি। `GET/POST/PUT /wc/v3/customers` আর `POST /wc/v3/customers/batch` সবই আছে, পুরো billing ও shipping address object সহ। দেখুন §13।

### স্কোপের বাইরে — এগুলো বানাবেন না

| কী                                             | কেন না                                                                                                                                                                                                                                                                                 |
|------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| আমাদের কার্টে লাইভ ERPNext Pricing Rule চালানো | দামটা ERPNext-এই হিসাব করে চূড়ান্ত সংখ্যাটা <code>regular_price</code> / <code>sale_price</code> হিসেবে পাঠান। তাহলে দাম হয়ে যায় এমন ডেটা যেটা আমাদের কাছে আগে থেকেই আছে। কোনো latency নেই, আপনি অফলাইন থাকলেও কাজ করে, আর কাস্টমারের কাছ থেকে ভুল টাকা নেওয়ার কোনো সুযোগ থাকে না। |
| Checkout-এ loyalty point redeem করা            | আলাদা প্রজেক্ট, আলাদা স্কোপ। এই কাজের অংশ না।                                                                                                                                                                                                                                          |
| Discount আর promotion-এর লজিক                  | একই কথা। কোনো ডিসকাউন্ট যদি স্টোরে পৌঁছাতেই হয়, সেটা আসবে দাম হিসেবে অথবা coupon হিসেবে — আমাদের checkout-এ লাইভ লজিক হিসেবে কখনোই না।                                                                                                                                                |
| আমাদের WordPress admin-এর ভেতরে একটা dashboard | মনিটরিং সেখানেই থাকবে যেখানে sync চলছে। সেটা আপনার দিক।                                                                                                                                                                                                                                |
| প্রোডাক্টের ছবি, ক্যাটাগরি, variation          | Phase 1-এ নেই। দরকার হলে আলাদাভাবে তুলবেন।                                                                                                                                                                                                                                             |

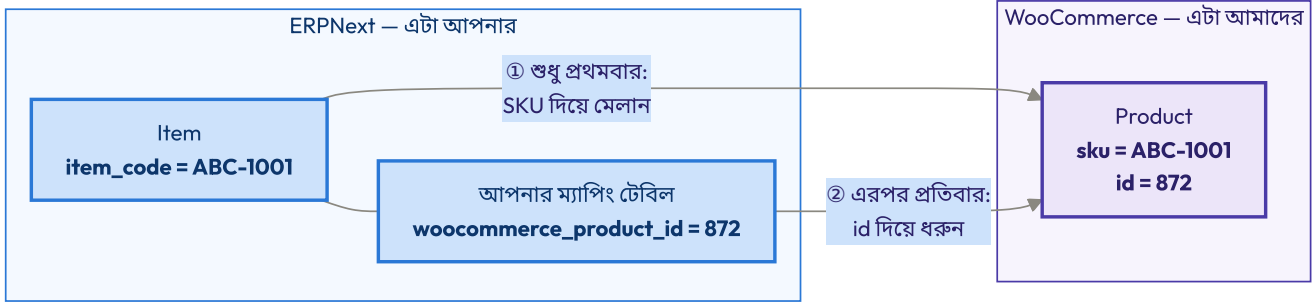
এর কোনোটা যদি মনে করেন আসলেই লাগবে, তাহলে বানানোর আগে বলুন। নিজে থেকে এই কাজের সাথে জুড়ে দেবেন না।

## ৪ দুই সিস্টেম কীভাবে একই জিনিস চিনবে

এটাই এই ডকুমেন্টের সবচেয়ে গুরুত্বপূর্ণ সেকশন। দুইবার পড়ুন।

আমরা যত sync সমস্যা দেখেছি, সবগুলোর মূলে একটাই ব্যাপার — কোন রেকর্ডটা কোনটা, তা নিয়ে দুই সিস্টেমের অমিল। এটা ঠিক কীভাবে কাজ করবে, তা এখানে।

### ৪.১ নিয়মটা: SKU হলো চুক্তি, WooCommerce ID হলো পয়েন্টার



ERPNext-এর `item_code` আর WooCommerce-এর `sku` -তে **হুবহু একই স্ট্রিং থাকবে**। এটাই আমাদের মধ্যে চুক্তি। এটা মানুষের পড়ার মতো, দুই স্ক্রিনেই দেখা যায়, আর কিছু গড়বড় হলে একজন মানুষ দুই সিস্টেম খুলে একই কোড দেখতে পায়।

কিন্তু আপনি আসলে ধরবেন WooCommerce-এর `id` ধরে। একবার SKU দিয়ে প্রোডাক্ট মিলিয়ে নেওয়ার পর তার WooCommerce `id` ERPNext-এ জমা রাখুন, আর এরপর প্রতিটা আপডেটে সেটাই ব্যবহার করুন। এটা সরাসরি lookup (`PUT /products/872`), কোনো search না — আর কখনো কোনো SKU সংশোধন করা হলেও এটা কাজ করতেই থাকে।

ম্যাপিং টেবিলটা পুরোপুরি ERPNext-এ থাকবে। আপনি আপনার Item রেকর্ডে `woocommerce_product_id` রাখবেন। আমরা কিছুই রাখব না। আমাদের প্রোডাক্টে কোনো custom field না, কোনো metadata না, কোনো লুকানো key না। আপনি জিজ্ঞেস করেছিলেন ERPNext-এর item code আমাদের স্টোরে রাখা উচিত কি না — উচিত না, আর শুধু আমাদের খাটুনি বাঁচাতে না। ইন্টিগ্রেশনের হিসাব-নিকাশ যে ইন্টিগ্রেট করছে তারই থাকার কথা। আপনার ম্যাপিং টেবিল কখনো হারিয়ে গেলে আমাদের প্রোডাক্টগুলো crawl করে SKU মিলিয়ে সেটা আবার বানিয়ে নিতে পারবেন। ওটা পাঁচ মিনিটের কাজ — আপনার state আমাদের ডেটাবেসে রাখার কারণ না।

### ৪.২ SKU কখনো বদলানো যাবে না

একবার SKU দেওয়া হয়ে গেলে সেটা স্থায়ী। কোনো প্রোডাক্টের সত্যিই অন্য কোড লাগলে সেটা **নতুন প্রোডাক্ট**, নাম বদলানো পুরোনো প্রোডাক্ট না।

এটাই আমাদের দিক থেকে দরকারি একমাত্র জিনিস, আর এটা আমরা মেনে চলব। এতে আমাদের কোনো খরচ নেই — আমরা শুধু SKU এডিট করব না। বিনিময়ে আপনার মেলানোর লজিকে rename-এর কেসটা কখনো সামলাতেই হবে না।

### ৪.৩ SKU আর barcode এক ফিল্ড না

আপনি বলেছেন item code, barcode আর SKU — তিনটাই একই মান হবে বলে আশা করছেন। কাজ চালানোর দিক থেকে তাতে সমস্যা নেই, আর নিজেদের ব্র্যান্ডের পণ্যের ক্ষেত্রে এটা স্বাভাবিকও। কিন্তু এগুলো **আলাদা ফিল্ড**, আর WooCommerce

এদের আলাদাভাবেই রাখে:

| ফিল্ড            | জিনিসটা কী                                                    | কে ঠিক করে                                       |
|------------------|---------------------------------------------------------------|--------------------------------------------------|
| sku              | আমাদের নিজেদের stock-keeping কোড। আমাদের স্টোরের ভেতরে ইউনিক। | আমরা। যা খুশি, আমাদের পছন্দমতো।                  |
| global_unique_id | Barcode — GTIN, UPC, EAN বা ISBN।                             | প্রস্তুতকারক / GS1। এটা আমাদের বানানোর জিনিস না। |

Product API-তে barcode রাখার জন্য WooCommerce-এ আলাদা করে `global_unique_id` ফিল্ড আছে। Barcode ওখানেই রাখবেন। আপনার ব্যবসা যদি চায় SKU আর barcode একই স্ট্রিং হোক, সেটা ব্যবসায়িক সিদ্ধান্ত আর সেটা কাজও করবে — কিন্তু তখন দুটো ফিল্ডেই লিখতে হবে, কারণ পরে যা কিছু barcode পড়ে (Google Shopping feed, marketplace listing, দোকানের scanner) সেগুলো `global_unique_id` পড়ে, `sku` না।

মেলানোর জন্য আমরা `sku` ব্যবহার করব। সবসময়। Barcode আমাদের বয়ে নিয়ে যাওয়া একটা ডেটা, মেলানোর চাবি না।

## 8.4 তিনটা জিনিসেই একই প্যাটার্ন

| জিনিস    | চুক্তি (মানুষের পড়ার মতো, দুই দিকেই)                    | পয়েন্টার (আপনি ERPNext-এ রাখবেন)                |
|----------|----------------------------------------------------------|--------------------------------------------------|
| Product  | <code>sku</code> $\equiv$ ERPNext <code>item_code</code> | WooCommerce product <code>id</code>              |
| Customer | <code>email</code>                                       | WooCommerce <code>customer_id</code>             |
| Order    | WooCommerce order <code>id</code>                        | — (উৎস আমরা; আপনি আমাদের <code>id</code> রাখবেন) |

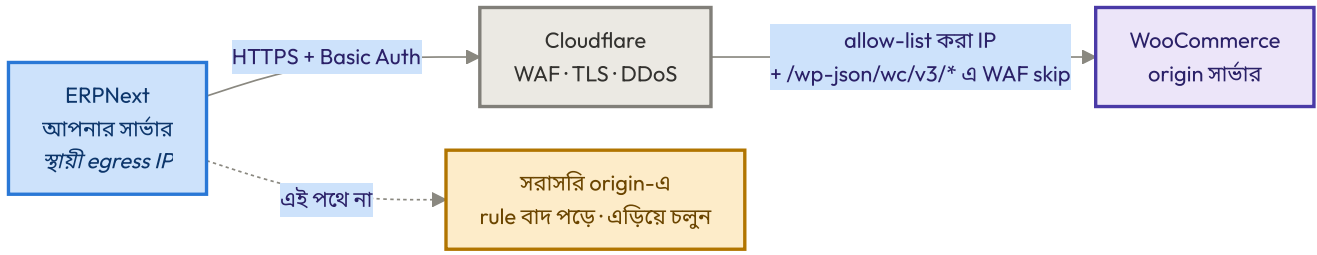
একটাই প্যাটার্ন, তিনটা জিনিসে। ব্যবসায়িক key দিয়ে একবার মেলান, তারপর চিরকাল ID দিয়ে ধরুন।

ERPNext-এ জমা রাখা প্রতিটা WooCommerce ID-এর উপর ডেটাবেস-লেভেলের unique index দিন।

অ্যাপ্লিকেশনের ভেতরের চেক না — আসল unique index। কোনো না কোনো সময় দুটো job একই মুহূর্তে একই অর্ডার import করার চেষ্টা করবেই, আর সেই দৌড়টা ঠিকমতো মেটাতে পারে শুধু ডেটাবেসই।

## 9 Cloudflare — আমরা এটা কীভাবে সামলাব

সংক্ষেপে: এটা আমাদের কাজ, আর আপনার আগেই এটা তৈরি হয়ে যাবে। এই সেকশনটা আছে যাতে আপনি জানেন কী ঘটছে আর আপনার কাছ থেকে ঠিক কোন জিনিসটা লাগবে — এখানে আপনার করার মতো কোনো কাজ আছে বলে না।



## বন্ধ না করে rule কেন

Cloudflare-এ মাঝামাঝি কোনো সেটিং নেই। একটা সুইচ টিপে এটাকে একটু ঢিলা করা, বা শুধু API ট্রাফিকের জন্য ঢিলা করা — কোনোটাই যায় না; টগলটা পুরো সাইটের জন্য হয় চালু নয় বন্ধ। আপনার কল ঢুকতে দেওয়ার জন্য এটা বন্ধ করা মানে এই মুহুর্তে আমাদের দিকে তাক করা প্রতিটা vulnerability scanner, credential-stuffing bot আর content scraper-এর জন্যও বন্ধ করা — আর একটা WooCommerce স্টোরে ওই ট্রাফিকের পরিমাণ নিয়মিত এবং মাপা যায় এমন।

তাই আমরা তার বদলে নির্দিষ্ট জিনিসটা করি: একটা **custom rule** যেটা আপনার ট্রাফিক চিনে সোজা ছেড়ে দেয়, আর সেটা বাঁধা থাকে আপনার IP এবং API path — দুটোর সাথেই। সাইটের বাকি সবকিছুতে পুরো প্রোটেকশন বহাল থাকে। সুইচ টেপার চেয়ে এটা আমাদের জন্য বেশি খাটুনি, সেজন্যই একটা-দুটো দিন চাইছি — কিন্তু লাইভে যাওয়ার পর নিরাপদে চালু রাখা যায় এমন সংস্করণ এটাই।

## আপনার কাছ থেকে যে একটা জিনিস লাগবে

যে IP address দিয়ে আপনার ERPNext সার্ভার বাইরে কল করে। §2-এর `cdn-cgi/trace` কমান্ড দিয়ে সার্ভার থেকেই যাচাই করা — DNS record দেখে না, কারণ একটা সার্ভার যে address থেকে পাঠায় সেটা প্রায়ই সেই address না যেটাতে সে গ্রহণ করে। কয়েকটা সার্ভার থাকলে CIDR range দিলেও চলবে।

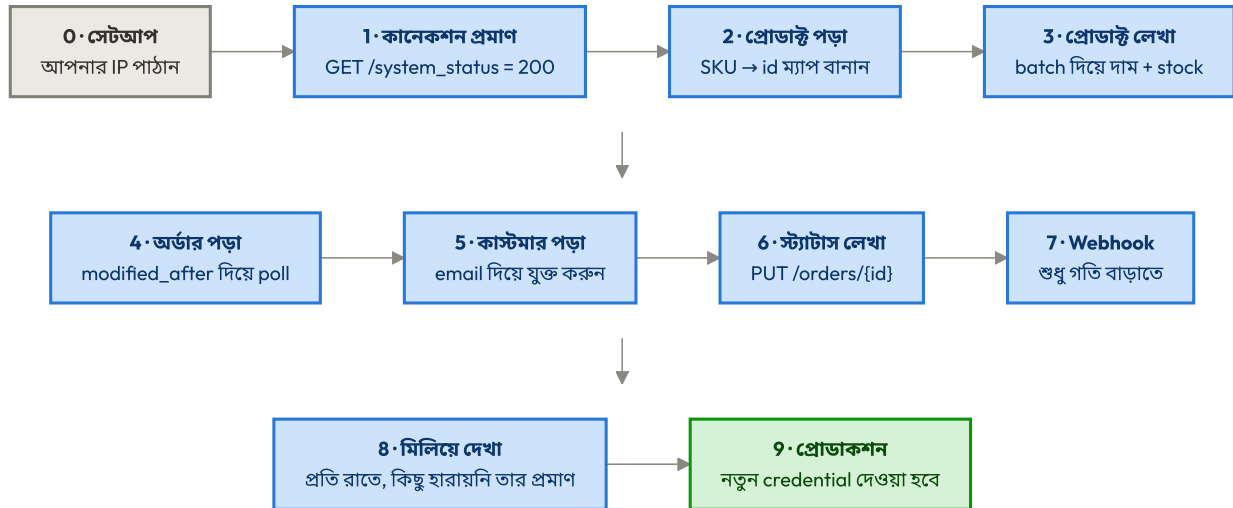
ওটুকু পাঠিয়ে দিন, বাকিটা আমরা করব। আপনার হোস্টিং যদি সত্যিই স্থায়ী outbound IP দিতে না পারে, আগেভাগে বলুন — আমরা তখন shared secret header ব্যবহার করব; ওটা একটু দুর্বল, তাই সাথে একটা rotation schedule থাকবে, কিন্তু কাজ করে আর কোনো সমস্যা না।

### কাজের কিছু কথা:

- **প্রতিটা request 100 সেকেন্ডের নিচে রাখুন।** দীর্ঘ সময় ধরে চলা request Cloudflare টাইমআউট করে দেয়।  $\leq 100$  আইটেমের ব্যাচ ব্যবহার করুন (এটা API-এর নিজের সীমাও) আর পেজিনেট করুন। এক কলে 5,000 প্রোডাক্ট পাঠানোর চেষ্টা করবেন না।
- **Webhook-এ এর কোনো প্রভাব নেই।** ওগুলো আমাদের সার্ভার থেকে *বাইরের দিকে* আপনার সার্ভারে যায়, তাই Cloudflare-এর inbound proxy ওই পথে থাকেই না।
- **403 বা কোনো challenge page পেলে** আমাদের পাঠান: সময়, source IP, পুরো URL আর response-এর `cf-ray` header। ওই Ray ID দেখে আমরা ঠিক বুঝতে পারব কোন rule-টা ধরেছে, আর কয়েক মিনিটেই ঠিক করে দিতে পারব। ঘুরপথ বের না করে জিনিসটা আমাদের পাঠিয়ে দিন — 403 প্রায় সবসময়ই আমাদের দিকের পাঁচ মিনিটের সমন্বয়।

## 10 কোন কাজটা কোন ক্রমে করবেন

এই ক্রমেই করবেন। আগে-পরে করবেন না। প্রতিটা ধাপ আলাদাভাবে টেস্ট করা যায়, আর প্রতিটা তার আগেরটার উপর নির্ভর করে।



**Webhook কেন ধাপ 7, ধাপ 1 না।** Webhook একটা গতি বাড়ানোর কৌশল। কোনো অর্ডার বদলালে সেটা কয়েক মিনিট পরে না জেনে কয়েক সেকেন্ড পরে জানায়। কিন্তু এটা নির্ভরযোগ্য delivery ব্যবস্থা না — delivery ফেল করে, retry হয়, এলোমেলো ক্রমে আসে, আর যে topic বারবার ফেল করে সেটা আপনাআপনি বন্ধ হয়ে যায়। **কিছু হারাচ্ছে না, সেটার নিশ্চয়তা দেয় polling।** আগে polling বানান, ঠিকঠাক করুন, তারপর latency কমাতে তার উপরে webhook বসান। এরপরেও polling চিরকাল চালু রাখবেন। Webhook আগে বানালে এমন একটা সিস্টেম পাবেন যেটা দেখতে দ্রুত, অথচ চুপচাপ অর্ডার ফেলে দেয়।

## 11 প্রোডাক্ট — ERPNext → WooCommerce

### 11.1 কোন ফিল্ড কোথায়

| ERPNext     | WooCommerce      | তৈরির সময় | আপডেটের সময় |
|-------------|------------------|------------|--------------|
| item_code   | sku              | ✓          | ✗ কখনো না    |
| barcode     | global_unique_id | ✓          | ✓            |
| item_name   | name             | ✓          | ✗            |
| description | description      | ✓          | ✗            |

| ERPNext           | WooCommerce                        | তৈরির সময় | আপডেটের সময় |
|-------------------|------------------------------------|------------|--------------|
| price             | <code>regular_price</code>         | ✓          | ✓            |
| discounted price  | <code>sale_price</code>            | ✓          | ✓            |
| stock qty         | <code>stock_quantity</code>        | ✓          | ✓            |
| —                 | <code>manage_stock ( true )</code> | ✓          | ✓            |
| হিসাব করে বের করা | <code>stock_status</code>          | ✓          | ✓            |

আপডেটের সময় শুধু দাম আর **stock**-এর ফিল্ডগুলো পাঠাবেন। WooCommerce আংশিক আপডেট করে — আপনি যা পাঠাবেন না, সেটা অপরিবর্তিত থাকবে, আর আমরা ঠিক সেটাই চাই।

আপডেটে কখনো `status` পাঠাবেন না। পুরোনো প্লাগিনটা প্রতিটা sync-এ `"status": "publish"` পাঠাত, যার ফলে আমাদের টিম ইচ্ছে করে draft করে রাখা প্রোডাক্টগুলো আবার publish হয়ে যেত। কোনো আইটেম বন্ধ করে দিলে পাঠাবেন `{"stock_status": "outofstock", "stock_quantity": 0}` — ডিলিট করবেন না, কারণ ডিলিট করলে ওই প্রোডাক্ট ধরে থাকা পুরোনো অর্ডারগুলো ভেঙে যায়।

দাম স্ট্রিং হিসেবে যাবে: `"1250.00"`, `1250.0` না। মেয়াদ শেষ হওয়া sale মুছতে পাঠাবেন `"sale_price": ""`।

## 11.2 পাঠানো

```
payload = {
  "update": [
    {"id": 872, "regular_price": "1180.00", "sale_price": "",
      "stock_quantity": 12, "stock_status": "instock"},
    {"id": 873, "regular_price": "640.00",
      "stock_quantity": 0, "stock_status": "outofstock"},
  ]
}

r = requests.post(f"{BASE}/products/batch",
                  json=payload, auth=AUTH, timeout=60)
```

প্রতি request-এ সর্বোচ্চ 100টি আইটেম।

একটা batch HTTP 200 ফেরত দিয়েও ভেতরের আলাদা আইটেম ফেল করতে পারে। Response-এর প্রতিটা element-এ `error` key আছে কি না দেখুন। এই endpoint-এ এটাই সবচেয়ে বেশি হওয়া ভুল — request সফল হয়, অথচ তিনটা প্রোডাক্ট চুপচাপ আপডেট হয় না, আর এক মাস ধরে কেউ টেরই পায় না।

```
for op in ("create", "update"):
    for item in r.json().get(op, []):
        if "error" in item:
            log.error("product %s failed: %s", op, item["error"])
```

### 11.3 প্রথমবার মেলানো

```
r = requests.get(f"{BASE}/products", params={"sku": "ABC-1001"}, auth=AUTH)
found = r.json()
# এই id-টা ERPNext-এ স্থায়ীভাবে জমা রাখুন
woo_id = found[0]["id"] if found else None
```

## 12 অর্ডার — WooCommerce → ERPNext

### 12.1 কোন অর্ডারগুলো নেবেন

| স্ট্যাটাস                         | যা করবেন                                                               |
|-----------------------------------|------------------------------------------------------------------------|
| pending , checkout-draft , failed | বাদ দিন। এটা বিক্রি না।                                                |
| processing                        | <b>Sales Order তৈরি করুন।</b> এটাই ট্রিগার।                            |
| completed                         | আগে না দেখে থাকলে তৈরি করুন; fulfilled মার্ক করুন।                     |
| cancelled                         | ERPNext ডকুমেন্টটা বাতিল করুন।                                         |
| refunded                          | Credit note / return।                                                  |
| on-hold                           | <b>আমাদের জিজ্ঞেস করুন</b> — এটা আমাদের পেমেন্ট মেথডের উপর নির্ভর করে। |

### 12.2 Polling

```
params = {
    "modified_after": high_water_minus_5_minutes, # ISO, UTC
    "dates_are_gmt": "true",
    "status": "processing,completed,cancelled,refunded",
```



```

"orderby": "modified", "order": "asc",
"per_page": 100,
}

```

নিয়ম:

- **High-water mark ERPNext-এ জমা রাখুন** (অর্থাৎ কোন সময় পর্যন্ত ডেটা নেওয়া হয়ে গেছে তার চিহ্ন)। মেমোরিতে না, আর redeploy-এ মুছে যায় এমন ফাইলেও না।
- **সফলভাবে লেখা হওয়ার পরেই সেটা এগোবেন।** লেখা ফেল করলে চিহ্নটা সরাবেন না।
- **প্রতিবার 5 মিনিট ওভারল্যাপ রাখুন**, যাতে আমাদের দুই সার্ভারের ঘড়ির পার্থক্য সামলে নেওয়া যায়। এতে ঝুঁকি নেই, কারণ আপনার import idempotent (একই জিনিস বারবার এলেও ফল একই থাকে)।
- **সবসময় `_gmt` তারিখের ফিল্ডগুলো আর `dates_are_gmt=true` ব্যবহার করুন।** স্থানীয় সময়েরগুলো কখনো না।

প্রতি 5-15 মিনিটে একবার চালাবেন।

## 12.3 যে ফিল্ডগুলো গুরুত্বপূর্ণ

```

{
  "id": 1042, "status": "processing", "currency": "BDT",
  "date_created_gmt": "2026-09-16T04:22:11",
  "total": "2680.00", "total_tax": "180.00", "discount_total": "100.00",
  "customer_id": 17, "prices_include_tax": false,
  "billing": { "first_name": "...", "email": "...", "phone": "...",
    "address_1": "...", "city": "...", "country": "BD" },
  "shipping": { "first_name": "...", "address_1": "...", "city": "...", "country": "BD" },
  "line_items": [
    { "product_id": 872, "sku": "ABC-1001", "quantity": 2,
      "subtotal": "2500.00", "total": "2400.00", "total_tax": "180.00" }
  ],
  "shipping_lines": [ { "method_title": "Flat rate", "total": "60.00" } ],
  "fee_lines": [],
  "coupon_lines": [ { "code": "WELCOME10", "discount": "100.00" } ]
}

```

একটা line item-এ `subtotal` হলো ডিসকাউন্টের আগের, আর `total` হলো পরের। হিসাবে `total` তুলবেন। `subtotal` তুললে আপনার আয়ের হিসাব ফুলে যাবে। এরপর সেভ করার আগে দেখে নিন আপনার হিসাব করা টোটাল (line + shipping + fee + tax) অর্ডারের `total` -এর সমান হচ্ছে কি না। না মিললে **সেভ করবেন না** — log করুন আর অ্যালার্ট দিন।

টাকার সব মান স্ট্রিং। এগুলো decimal হিসেবে পার্স করবেন, float হিসেবে কখনো না। `currency` ধরে নেবেন না, অর্ডার থেকেই পড়ুন।

## 12.4 Idempotency

```
existing = frappe.db.get_value(
    "Sales Order", {"woocommerce_order_id": order["id"]}, "name")
if existing:
    update_sales_order(existing, order)
else:
    create_sales_order(order)
```

সাথে `woocommerce_order_id` -এর উপর একটা ডেটাবেস **unique index**, আর দুটো worker একসাথে পড়লে duplicate-key error ধরার ব্যবস্থা। আমরা এটা টেস্ট করব একই অর্ডার পাঁচবার replay করে — আশা করব ঠিক একটা Sales Order।

## 12.5 স্ট্যাটাস ফেরত পাঠানো

```
requests.put(f"{BASE}/orders/1042", json={"status": "completed"}, auth=AUTH)
```

**লুপ থেকে বাঁচুন।** আপনার স্ট্যাটাস আপডেট `date_modified_gmt` বদলে দেয়, তাতে আপনার নিজের poller অর্ডারটা আবার তুলে নেয়, তাতে আবার একটা আপডেট হতে পারে। শেষবার কোন স্ট্যাটাস পাঠিয়েছিলেন সেটা মনে রাখুন, আর নতুন আসা মানটা তার সমান হলে বাদ দিন। এই রক্ষাকবচটা না থাকলে দুই সিস্টেমের মধ্যে একটা অসীম লুপ তৈরি করবেন — আর প্রতিটা চক্র আলাদাভাবে দেখতে একদম নিখুঁত লাগবে।

# 13 কাস্টমার

কাস্টমার দুইভাবে আসে, আর কোনটা কোনভাবে এসেছে সেটা গুরুত্বপূর্ণ:

**অর্ডারের সাথে।** প্রতিটা অর্ডারে পুরো `billing` আর `shipping` ব্লক থাকে। গেস্ট checkout-এর ক্ষেত্রে (`customer_id: 0`) এটুকুই থাকে, আর Sales Order তোলার জন্য এটুকুই যথেষ্ট। **প্রতিটা গেস্ট অর্ডারের জন্য আলাদা ERPNext Customer রেকর্ড বানাবেন না** — আগে email দিয়ে ডুপ্লিকেট বাদ দিন, নইলে একজন বারবার কেনা গেস্ট ক্রেতা পঞ্চাশটা Customer রেকর্ড হয়ে যাবে।

**অ্যাকাউন্টধারী হিসেবে।** `customer_id` শূন্যের বড় হলে বুঝবেন সত্যিকারের একটা WooCommerce কাস্টমার অ্যাকাউন্ট আছে, যেটা আপনি পড়তে আর সিঙ্গে রাখতে পারবেন:

```
customer = requests.get(f"{BASE}/customers/17", auth=AUTH).json()
```

যা যা আছে: `GET /customers` (লিস্ট), `GET /customers/{id}`, `POST /customers`, `PUT /customers/{id}`, আর একবারে 100টা পর্যন্ত করার জন্য `POST /customers/batch`। প্রতিটা রেকর্ডে থাকে `email`, `first_name`, `last_name`, `username`, `role`, `is_paying_customer`, আর পুরো `billing` ও `shipping` object।

`email` দিয়ে মেলান, `customer_id` জমা রাখুন — প্রোডাক্টের মতো একই প্যাটার্ন।

এটা সহজ রাখুন। কাস্টমার sync মানে: আমাদের প্রতিটা অ্যাকাউন্টধারীর জন্য ERPNext-এ একটা Customer আছে, আর তার যোগাযোগের তথ্য হালনাগাদ। এর মানে দুই দিকের ঠিকানার খাতা মিলিয়ে দেওয়ার কোনো ব্যবস্থা বানানো না। ERPNext থেকে কোনো কাস্টমারের ঠিকানা শুধরাতে হলে `PUT /customers/{id}` আছে, কিন্তু কাস্টমারের পরিচয়ের ক্ষেত্রে WooCommerce-কেই আসল উৎস ধরবেন।

## 14 চালানোর নিয়ম

**ঠিকমতো retry করুন।** 429, 500, 502, 503, 504 আর connection error-এর ক্ষেত্রে ব্যাক-অফ দিয়ে retry করুন — 1s, 2s, 4s, 8s, 16s, তারপর থেমে যান। `Retry-After` থাকলে সেটা মানুন। 400, 401, 403 বা 404 -এ retry করবেন না — ওগুলো আপনার request-এর ভুল, retry করলে শুধু আমাদের উপর চাপ পড়ে। ওগুলো log করে থেমে যান।

**Rate limit।** প্রোডাকশনে মোটামুটি মিনিটে 60টা request-এর নিচে থাকুন। Batch endpoint ব্যবহার করুন। বড় reconciliation চালাবেন স্টোরের স্থানীয় সময় 09:00–23:00-এর বাইরে। বড় কোনো একবারের backfill লাগলে আগে আমাদের বলুন, যাতে চলার সময় আমরা স্টোরটা নজরে রাখতে পারি।

**Credential।** ERPNext-এর site config বা environment variable-এ রাখুন। কখনো source control-এ না, কখনো log-এ না, কখনো screenshot-এ না, কখনো সাপোর্ট টিকিটে না। TLS সার্টিফিকেট যাচাই কখনো বন্ধ করবেন না — `verify=False` না, `curl -k` না, ডিবাগ করার সময় সাময়িকভাবেও না।

**Logging।** প্রতিটা sync অপারেশন log করুন: সময়, কোন দিকে, কোন জিনিস, দুই দিকের ID, HTTP status, ফলাফল। Credential মুছে দিন। আমরা একসাথে এভাবেই ডিবাগ করব, আর আগে আমাদের admin-এ যে dashboard ছিল এটা তার জায়গা নেবে।

**প্রতি রাতে মিলিয়ে দেখুন।** একই সময়সীমার জন্য WooCommerce-এ যত অর্ডার আছে আর ERPNext-এ যত আছে, দুটো মিলিয়ে দেখুন। আপনার দিকে কিছু না থাকলে সেটা প্রোডাকশন ইনসিডেন্ট — অ্যালার্ট দিন, শুধু একটা log লাইন লিখে রাখবেন না।

**ভেঙে গেলে আমাদের জানান।** Credential ফাঁস হলে, অ্যাক্সেস আছে এমন কেউ চলে গেলে, বা ব্যাখ্যা করতে পারছেন না এমন API response দেখলে — 24 ঘণ্টার মধ্যে।

## 15 অ্যাকসেসপেটস চেকলিস্ট

এগুলোর প্রতিটা আমরা টেস্ট করব। কোনো ধাপের প্রতিটা ঘরে টিক না পড়া পর্যন্ত সেই ধাপ শেষ হয়নি।

### রেডিনেস চেক — আমাদের কাছে কিছু চাওয়ার আগে

- ☐ নিজের stock WooCommerce-এ 10টা টেস্টই চালানো হয়েছে, আর আসল আউটপুট আমাদের পাঠানো হয়েছে
- ☐ কোন WooCommerce আর WordPress ভার্সনে টেস্ট করেছেন, লিখিতভাবে জানানো হয়েছে
- ☐ কোনো পর্যায়েই কোনো WordPress প্লাগিন বানানো হয়নি

### কানেকশন

- ☐ GET /system\_status 200 দিচ্ছে
- ☐ সব ট্রাফিক Cloudflare দিয়ে যাচ্ছে, সরাসরি origin-এ না
- ☐ সব জায়গায় সার্টিফিকেট যাচাই চালু আছে
- ☐ Credential source control-এ নেই

### প্রোডাক্ট

- ☐ প্রতিটা ERPNext Item-এ woocommerce\_product\_id জমা আছে
- ☐ প্রথমবার SKU দিয়ে মেলানো হচ্ছে, এরপর থেকে ID দিয়ে
- ☐ আপডেটে শুধু দাম আর stock যাচ্ছে — কখনো status , name বা description না
- ☐ দাম স্ট্রিং হিসেবে যাচ্ছে; মেয়াদ শেষ sale মুহুর্তে sale\_price: "" যাচ্ছে
- ☐ Batch ≤100, আর response-এর প্রতিটা element-এ error আছে কি না দেখা হচ্ছে
- ☐ পুরো sync দুইবার চালালে দ্বিতীয়বারে কিছুই বদলায় না

### অর্ডার

- ☐ woocommerce\_order\_id -এর উপর ডেটাবেস unique index আছে
- ☐ High-water mark restart-এর পরেও টিকে থাকে, আর সফলভাবে লেখার পরেই এগোয়
- ☐ Polling-এ 5 মিনিট ওভারল্যাপ আছে; প্রতিটা query-তে dates\_are\_gmt=true
- ☐ Line-এর total হিসাবে তোলা হচ্ছে, subtotal না
- ☐ সেভ করার আগে হিসাব করা টোটাল অর্ডারের total -এর সাথে মিলিয়ে দেখা হচ্ছে
- ☐ গেস্ট অর্ডার ( customer\_id: 0 ) ডুপ্লিকেট Customer বানাচ্ছে না
- ☐ স্ট্যাটাস ফেরত পাঠানোয় লুপ থেকে বাঁচার ব্যবস্থা আছে

### Webhook (ধাপ 7)

- ☐ Signature যাচাই হচ্ছে raw request body-র বিপরীতে, hmac.compare\_digest দিয়ে

- ☐ ভুল signature-এ 401 ফেরত যায় আর আর কিছুই হয় না
- ☐ 5 সেকেন্ডের কমে 200 ফেরত দেয়; কাজটা queue-তে যায়, সাথে সাথে করা হয় না
- ☐ Handler payload-কে বিশ্বাস না করে API থেকে অর্ডারটা আবার এনে নেয়
- ☐ একটা webhook 5 বার replay করলে ঠিক একটা Sales Order হয়

#### অপারেশন

- ☐ 429/5xx-এ exponential backoff; 4xx-এ কোনো retry নেই
- ☐ প্রতিটা অপারেশন দুই দিকের ID সহ log হচ্ছে
- ☐ রাতের reconciliation চলছে আর কিছু বাদ পড়লে অ্যালার্ট দিচ্ছে

## 16 কিছু কাজ না করলে

আমাদের বলুন। নিজে থেকে ঘুরপথ বের করবেন না।

আপনার দিকের একটা ঘুরপথ আমাদের দুই সিস্টেমের মধ্যে স্থায়ী, কোথাও লেখা নেই এমন একটা পার্থক্য তৈরি করে ফেলে — আর ঠিক এই জিনিসগুলোই একটা ইন্টিগ্রেশনকে পরে আর রক্ষণাবেক্ষণযোগ্য রাখে না। স্টেজিং স্টোর আসলে যা করছে সেটা যদি এই স্পেকের সাথে না মেলে, তাহলে সেটা আমাদের স্পেকের ভুল আর আমরা সেটা জানতে চাই।

কয়েকটা জিনিস আপনার ঘুরে যাওয়ার চেয়ে আমাদের ঠিক করে দেওয়া দ্রুত — এগুলোর কোনোটার দিকে এগোচ্ছেন মনে হলে গ্রুপে মেসেজ দিন, আমরা সামলে দেব:

| আপনি যেটা করতে যাচ্ছেন...                                     | তার বদলে আমাদের এটা পাঠান                                                                    |
|---------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Cloudflare proxy বন্ধ করতে বলা                                | Block হওয়া response-এর <code>cf-ray</code> header — আমরা rule ঠিক করে দেব, সাধারণত একই দিনে |
| সরাসরি origin IP-তে কানেক্ট করা                               | একই জিনিস — সরাসরি origin-এ গেলে আপনার জন্য বানানো rule বাদ পড়ে যায় আর পরে ভেঙে পড়ে       |
| TLS সার্টিফিকেট যাচাই বন্ধ করা                                | সার্টিফিকেট error-এর লেখাটা                                                                  |
| একটা WordPress প্লাগিন বানানো                                 | আপনি কী করতে চাইছেন তার বর্ণনা — ওটার জন্য API-তে পথ আছে                                     |
| আপনার ম্যাপিংয়ের জন্য আমাদের প্রোডাক্টে custom field যোগ করা | কিছুই না — ম্যাপিং ERPNext-এই রাখুন, §8 দেখুন                                                |
| আমাদের checkout-এর পথে কিছু বসানো                             | কোন ঘটনাটা আপনার দরকার সেটা লিখে পাঠান; প্রায় নিশ্চিতভাবেই webhook দিয়ে করা যাবে           |

একটা প্রশ্নের দাম গ্রুপে একটা মেসেজ। একটা ভুল ধারণা লাইভে যাওয়ার দিনে ধরা পড়লে তার দাম পুরোটা আবার বানানো।